

```
import tensorflow as tf
from tensorflow.keras.layers import Dense, Conv2D, MaxPool2D, Flatten
import matplotlib.pyplot as plt
import numpy as np
```

```
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.cifar10.load_data()
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 — 3s 0us/step

```
# Normalise dataset
train_images=train_images/255.0
test_images=test_images/255.0
```

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
for i in range(5):
    plt.imshow(train_images[i])
    plt.title(class_names[train_labels[i][0]])
    plt.axis('off')
    plt.show()
```



```
#Build cnn
model=tf.keras.Sequential()
# Add an Input layer as suggested by the warning
model.add(tf.keras.Input(shape=(32,32,3)))
#convolutional layer 1
#32 filter
# 3x3 kernal size
#relu= activation function
model.add(Conv2D(32,(3,3),activation='relu')) # Removed input_shape here
model.add(MaxPool2D((2,2)))

model.add(MaxPool2D((2,2)))
#conv layer 2
model.add(Conv2D(64,(3,3),activation='relu'))
model.add(MaxPool2D((2,2)))
# flatten+ fully connector layers
model.add(Flatten())

#Hidden layers
model.add(Dense(64,activation='relu'))
#output layer
model.add(Dense(10,activation='softmax'))
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 32)	0
conv2d_1 (Conv2D)	(None, 5, 5, 64)	18,496
max_pooling2d_2 (MaxPooling2D)	(None, 2, 2, 64)	0
flatten (Flatten)	(None, 256)	0
dense (Dense)	(None, 64)	16,448
dense_1 (Dense)	(None, 10)	650

Total params: 36,490 (142.54 KB)
 Trainable params: 36,490 (142.54 KB)
 Non-trainable params: 0 (0.00 B)

```
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=['accuracy']
)
model.fit(train_images,train_labels,epochs=10,validation_data=(test_images,test_labels))
```

```
Epoch 1/10
1563/1563 — 14s 4ms/step — accuracy: 0.3737 - loss: 1.7055 - val_accuracy: 0.4688 - val_loss: 1.4546
Epoch 2/10
1563/1563 — 6s 4ms/step — accuracy: 0.5072 - loss: 1.3788 - val_accuracy: 0.5352 - val_loss: 1.3056
Epoch 3/10
1563/1563 — 7s 4ms/step — accuracy: 0.5507 - loss: 1.2630 - val_accuracy: 0.5610 - val_loss: 1.2340
Epoch 4/10
1563/1563 — 10s 4ms/step — accuracy: 0.5787 - loss: 1.1875 - val_accuracy: 0.5832 - val_loss: 1.1779
Epoch 5/10
1563/1563 — 10s 4ms/step — accuracy: 0.5979 - loss: 1.1377 - val_accuracy: 0.6012 - val_loss: 1.1271
Epoch 6/10
1563/1563 — 6s 4ms/step — accuracy: 0.6124 - loss: 1.0987 - val_accuracy: 0.6090 - val_loss: 1.1086
Epoch 7/10
1563/1563 — 6s 4ms/step — accuracy: 0.6264 - loss: 1.0589 - val_accuracy: 0.6093 - val_loss: 1.1142
Epoch 8/10
```